

P4205R0

Range-Based Searchers

Published Proposal, 2026-04-29

This version:

<https://wg21.link/P4205R0>

Author:

[Yihe Li](#)

Audience:

SG9

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:

C++29

Source:

github.com/Mick235711/wg21-papers/blob/main/P4205/P4205R0.bs

Issue Tracking:

[GitHub Mick235711/wg21-papers](#)

Table of Contents

- 1 Revision History**
 - 1.1 R0 (2026-05 pre-Brno mailing)
- 2 Background**
- 3 Motivation**
- 4 Design**
 - 4.1 What Is Being Proposed?
 - 4.2 Range-Based Searchers
 - 4.3 Building A Searchers Concept

- 4.4 `constexpr` And Return Type
- 4.5 Header Choice
- 4.6 Feature-Test Macro Changes
- 4.7 Freestanding
- 4.8 Alternatives Considered
- 4.9 Future Extensions
 - 4.9.1 Parallel Overloads
 - 4.9.2 Searcher Overloads In More Algorithms
 - 4.9.3 More Searchers

5 Implementation Experience

6 Questions To Resolve

- 6.1 Questions for SG9
- 6.2 Questions for LEWG

7 Wording

- 7.1 17.3.2 Header `<version>` synopsis [version.syn]
- 7.2 22.10.2 Header `<functional>` synopsis [functional.syn]
- 7.3 22.10.18.5 Range-based searchers [func.range.search]
 - 7.3.1 22.10.18.5.1 Class template `default_searcher` [func.range.search.default]
 - 7.3.2 22.10.18.5.2 Class template `boyer_moore_searcher` [func.range.search.bm]
 - 7.3.3 22.10.18.5.3 Class template `boyer_moore_horspool_searcher` [func.range.search.bmh]
- 7.4 24.2 Header `<iterator>` synopsis [iterator.synopsis]
- 7.5 24.3.7.❖ Concept `searchable` [alg.req.searchable]
- 7.6 26.4 Header `<algorithm>` synopsis [algorithm.syn]
- 7.7 26.6.4 Contains [alg.contains]
- 7.8 26.6.15 Search [alg.search]

References

Normative References

Informative References

This proposal introduces `std::ranges` versions of the `Searcher` overload of the `std::search` algorithm, which takes a searcher object instead of an iterator pair and (optionally) a predicate. It was originally introduced for more performant specialized searching.

As customary in Ranges algorithms, this proposal also proposes Range-ified versions of the existing standard searchers, along with a concept `std::searchable` for better capturing the semantic requirements of standard searchers.

§ 1. Revision History

§ 1.1. R0 (2026-05 pre-Brno mailing)

- Initial revision.

§ 2. Background

`std::search` is a very old standard algorithm, dating back to the inclusion of the original STL in 1994. The most basic shape of the algorithm takes two pairs of iterators representing haystack and needle, and returns the beginning position of the first occurrence of the needle sequence in the haystack sequence (or returns the end iterator of haystack if no occurrences are found). Optionally, the algorithm also takes a binary predicate to overwrite the equality comparison operation. The algorithm also has a `search_n` variant that searches for `count` copies of the same element instead of a provided needle range.

Several improvements and extensions have been applied to `std::search` after its inclusion in the C++ standard. In the C++17 cycle, the introduction of parallel algorithms brings `ExecutionPolicy` overloads to `std::search`; in the C++20 cycle alone, the merging of Ranges TS ([P0896R4]) into the standard brings with it the `std::ranges::search` variant, which enables iterator-sentinel and ranges argument passing style instead of the traditional iterator pair style to the algorithm. Meanwhile, [P0202R3] also added the `constexpr` specifier to both variants of the algorithm, finally leading to its present form.

However, all of these modifications apply uniformly to most algorithms in the `<algorithm>` header; this proposal instead focuses on one specific enhancement that only applies to `std::search`: the `Searcher` overload of the algorithm introduced by [N3905] (originally to Library Fundamentals TS v1, later merged into C++17). The original motivation for the inclusion of searchers was to allow more performant specialized search algorithms that precompute various statistics from the pattern to increase performance, as `std::search`'s normal overloads in general can only achieve a time complexity of $O(NS)$ using naive methods (where N and S are the length of the haystack and needle sequences, respectively). For instance, the standard `boyer_moore_searcher` implements the [Boyer-Moore String Search Algorithm](#), which can achieve a worst-case running time of $O(N + S)$ instead (requires some variation on the original 1977 version of the algorithm). In practice, using standard

searchers often introduces [significant speedup](#) compared to normal `std::search` invocations, at the cost of a larger memory footprint and longer initialization times on first invocation due to the need for precomputations.

Unfortunately, perhaps due to its late entrance to the standard, the `Searcher` overload is never included in either `Boost.Ranges` or `range-v3`, and as a result, the current Ranges library never included a `std::ranges::search` overload with `Searcher` arguments. Given that nearly every other overload in the `<algorithm>` header has a corresponding `ranges` version, and works ([P1813R0], [P3732R2]) have been underway to Range-ify the remaining `<numeric>` algorithms too, it seems increasingly likely that the omission of this overload is an unintentional mistake.

This proposal thus proposes fixing this omission by introducing an overload for `std::ranges::search` that accepts a `Searcher` algorithm instead of a second range. Furthermore, the C++23 algorithm `std::ranges::contains_subrange` introduced by [P2302R4] was specified as a convenience wrapper for `std::ranges::search`, thus a `Searcher` overload is introduced for that algorithm too for API consistency.

§ 3. Motivation

Current inconsistency in the `std::ranges::search` API forces users to exit the Ranges world to use searchers and resort to using traditional STL algorithms instead, which is undesirable.

A before/after comparison of the feature is shown below:

```
// In C++26
std::string haystack = "a quick brown fox jumps over the lazy dog";
std::string needle = "jump";

std::search(haystack.begin(), haystack.end(), needle.begin(), needle.end(),
std::ranges::search(haystack, needle); // OK, clear improvements to std::search

std::search(haystack.begin(), haystack.end(), needle.begin(), needle.end(),
std::ranges::search(haystack, needle, [](auto a, auto b) { return tolower(a); },
std::ranges::search(haystack, needle, {}, [](auto a) { return tolower(a); });

std::search(std::execution::par_unseq, haystack.begin(), haystack.end(), needle.begin(), needle.end(),
std::ranges::search(std::execution::par_unseq, haystack, needle); // OK since C++23

std::boyer_moore_searcher searcher(needle.begin(), needle.end());
std::search(haystack.begin(), haystack.end(), searcher); // OK
```

```
std::ranges::search(haystack, needle, searcher); // Error!

// After this proposal:
std::ranges::boyer_moore_searcher searcher(needle);
std::ranges::search(haystack, needle, searcher); // Proposed OK
```

Since most Ranges algorithms use concepts to constrain their arguments instead of using template parameter names as an implicit named requirement, this proposal also follows this custom and introduces a concept for searchers that constraints their API, instead of relying on named requirements.

§ 4. Design

§ 4.1. What Is Being Proposed?

- The following additional overloads for `std::ranges::search` and `std::ranges::contains_subrange`:

```
template<forward_iterator I, sentinel_for<I> S, class Searcher, class Proj>
    requires searchable<Searcher, I, S, Proj>
constexpr auto ranges::search(I first, S last, const Searcher& searcher, Proj proj) {
template<forward_range R, class Searcher, class Proj = identity>
    requires searchable<Searcher, iterator_t<R>, sentinel_t<R>, Proj>
constexpr auto ranges::search(R&& r, const Searcher& searcher, Proj proj) {

template<forward_iterator I, sentinel_for<I> S, class Searcher, class Proj>
    requires searchable<Searcher, I, S, Proj>
constexpr bool ranges::contains_subrange(I first, S last, const Searcher& searcher, Proj proj) {
template<forward_range R, class Searcher, class Proj = identity>
    requires searchable<Searcher, iterator_t<R>, sentinel_t<R>, Proj>
constexpr bool ranges::contains_subrange(R&& r, const Searcher& searcher, Proj proj) {
```

- Concept `std::searchable` that captures the standard searcher requirements
- `std::ranges::default_searcher`, `std::ranges::boyer_moore_searcher`, `std::ranges::boyer_moore_horspool_searcher`

§ 4.2. Range-Based Searchers

The simplest version of a standard searcher is realized in `std::default_searcher`, which acts as a thin wrapper around `std::search` to make it follow the searcher API. The API for `std::default_searcher` looks like this:

```
template<class ForwardIterator1, class BinaryPredicate = equal_to<>>
class default_searcher {
public:
    constexpr default_searcher(ForwardIterator1 pat_first, ForwardIterator1 pat_last,
                               BinaryPredicate pred = BinaryPredicate());

    template<class ForwardIterator2>
    constexpr pair<ForwardIterator2, ForwardIterator2>
    operator()(ForwardIterator2 first, ForwardIterator2 last) const;
};
```

There are several peculiarities in this API that make it unsuitable for the Ranges library, thus prompting this proposal to propose new `std::ranges` versions of the standard searchers, similarly to the relation between `std::ranges::less` and `std::less<T>`. For instance, all standard searchers' `operator()` returns a pair of iterators after [P0253R1] identified that the end position of the needle inside the haystack is frequently useful for algorithms that are based on searchers. The Ranges library introduced `std::ranges::subrange` directly for this purpose, and thus, that should be used as the return type instead.

The proposed API for `std::ranges::default_searcher` looks like this:

```
template<forward_range R, copy_constructible Pred = ranges::equal_to,
        indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity>
class ranges::default_searcher {
public:
    constexpr default_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_last,
                               Pred pred = Pred(), Proj proj = Proj());

    template<typename R2> requires std::same_as<std::remove_reference_t<R2>, R>
    constexpr explicit default_searcher(R2&& r);

    template<typename R2> requires std::same_as<std::remove_reference_t<R2>, R>
    constexpr default_searcher(R2&& r, Pred pred, Proj proj = Proj());

    template<forward_iterator I2, sentinel_for<I2> S2, class Proj2 = identity>
    requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj>
    constexpr subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = Proj2()) const;

    template<forward_range R2, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R2>, iterator_t<R>, Pred, Proj2, Proj>
    constexpr subrange<R2> operator()(R2 first, R2 last, Proj2 proj2 = Proj2()) const;
```

```
constexpr borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = {})
};

template<std::ranges::forward_range R, std::copy_constructible Pred = std::
    std::indirectly_regular_unary_invocable<std::ranges::iterator_t<R>,
    default_searcher(R&&, Pred = {}, Proj = {}) -> default_searcher<std::remove
```

Essentially, the API for `std::ranges::default_searcher` is a Range-ified version of `std::default_searcher`, with the intention that `std::ranges::search(r, std::ranges::default_searcher(r2, pred, proj2), proj)` (which calls `searcher(r, proj)`) is always equivalent to `ranges::search(r, r2, pred, proj, proj2)`. It is for this reason that the concepts on `operator()` have its arguments flipped.

The API for `std::ranges::boyer_moore_[horspool_]searcher` adds a `Hash` type argument as the last template argument (that defaults to `std::hash<projected_value_t<iterator_t<R>, Proj>>`), and adds a `Hash hf = {}` argument to the constructor. Apart from these changes, their API is identical to the default searchers (except requiring random-access iterators and ranges instead of forward ones). The `Hash` argument is changed from the second argument to the last because of its default argument dependency on the projection. It is expected that most users will utilize CTAD to construct these searchers (i.e. `boyer_moore_searcher{needle}`), so the positions of template argument are not that important anyways.

One thing to be noted here is that the proposed searchers have a lifetime dependency on the underlying ranges; the standard searchers will store the iterators from the provided range, and thus is only valid for use while the provided range is alive (or if it is a borrowed range). The author initially considered requiring `borrowed_range<R>` to remove this safety hazard, but ultimately decided that allowing thing like `boyer_moore_searcher{some_string}` is much more important, as most searchers are used directly after construction anyways. The proposed searchers also have to store a copy of the predicate and projection, so these should ideally be cheap to copy.

§ 4.3. Building A Searchers Concept

The concept `std::searchable` itself does not fully capture the API proposed for the Range versions of searchers; this is because the proposed concept is iterator-based, so the Range constructor and Range-based `operator()` are not a required part of the searchers interface to reduce requirements on user-provided searcher types. However, range-accepting overloads of `std::ranges::search` will still call the searcher's range-accepting `operator()` if it is available, and fall back to iterator ver-

sions of `operator()` otherwise. This design enables optimization opportunities when the searcher can meaningfully treat a range different from its begin/end iterators.

The proposed specification for the `std::searchable` concept is as follows:

```
template<class Searcher, class I, class S, class P = identity>
concept searchable =
    copyable<Searcher> && // C++17 searchers require Cpp17CopyConstructible
    invocable<Searcher, I, S, P> &&
    ranges::forward_range<invoke_result_t<Searcher, I, S, P>>
```

Note that we deliberately use `invocable` here since most implementations of non-trivial searchers are stateful.

§ 4.4. `constexpr` And Return Type

It is worth noting that `boyer_moore_[horspool_]searcher` are *not* marked as `constexpr` currently, so the `std::ranges` versions also do not support compile-time evaluation. `default_searcher` on the other hand did support `constexpr`.

The return type for the new overloads of `std::ranges::search` is specified as `auto` (i.e., copy the return type of the searcher's `operator()`). It is possible to require convertible to `subrange<I>` here too, but the author thinks that this is an unnecessary limitation. The searcher's `operator()` return type is constrained to be a forward range by the concept `searchable`.

§ 4.5. Header Choice

The existing standard searchers live in `<functional>`, and all overloads of the `std::search` algorithm live in `<algorithm>`.

This proposal proposes that the new overloads of `std::ranges::search` and `std::ranges::contains_subrange` both still go into `<algorithm>`, which is consistent with other overloads of the two algorithms. The `std::ranges` versions of the three existing standard searchers should also still go into `<functional>`, following the precedent of `ranges::less`. The concept `std::searchable` will go into `<iterator>`, similar to other `indirectly_foo` and `sortable` concepts.

§ 4.6. Feature-Test Macro Changes

Currently, both `boyer_moore_searcher` and `boyer_moore_horspool_searcher` use the same feature-test macro `__cpp_lib_boyer_moore_searcher`, and `default_searcher` does not have a corresponding macro (its `constexpr`-edness is tracked by `__cpp_lib_constexpr_functional`). Also, `std::ranges::search` does not have its own feature-test macro, and the author thinks that bumping `__cpp_lib_ranges` for this seems a bit too huge.

This proposal ultimately decided to propose the following changes to feature-test macros:

- A new FTM `__cpp_lib_ranges_searcher` for all Range versions of searchers
- Do not bump the existing `__cpp_lib_boyer_moore_searcher`; the author thinks that macro is more appropriate for direct functional changes to the searcher. This proposal also does not make any changes to the existing `std::boyer_moore_[horspool_]searcher` anyway
- A new FTM `__cpp_lib_ranges_search` for `std::ranges::search` (should track all overloads)
- Bump the existing `__cpp_lib_ranges_contains` FTM for `ranges::contains_subrange` (this FTM is also used by `ranges::contains` too)

Of course, the author is happy to split the FTMs further (such as separate FTMs for each Range searcher) if LEWG requests.

§ 4.7. Freestanding

After the adoption of [P2976R1] in C++26, the majority of `<algorithm>` (including all overloads of `std::search` and `std::ranges::search`) are marked as freestanding. This proposal does not change anything in that regard, so the newly proposed overloads are also available in freestanding.

The searchers have a somewhat different status. `std::boyer_moore_[horspool_]searcher` is deliberately left out of freestanding by [P1642R11] as their common implementations rely on dynamic allocation, so their range versions are also not marked freestanding.

`std::ranges::default_searcher` is marked freestanding as it is just a thin wrapper around `std::ranges::search` anyway. The concept `std::searchable` is marked freestanding to maintain consistency with other parts of `<iterator>`.

All feature-test macros added by this paper are marked freestanding, and all modified FTMs are already freestanding after the adoption of [P2198R7] and [LWG4286]. No existing features are moved to freestanding, so the freestanding-related FTMs are not changed.

§ 4.8. Alternatives Considered

A possible less invasive change is to reuse the existing searchers, but only update `std::ranges::search` with the new overloads. In that path, the `searchable` concept will probably accept only an iterator type and test for `forward_iterator_pair<invoke_result_t<Searcher, I, I>>`.

However, such a change will not support range-based construction and `operator()`, not support projections on needles, and also not adapt to the new C++20 range/iterator model. The author thinks that requiring the user to write `ranges::search(haystack, boyer_moore_searcher(needle.begin(), needle.end()))` is strictly worse than introducing Range-ified versions of the searchers.

It has also been considered briefly to add a default argument to the `Searcher` argument of the algorithms; however, using `default_searcher` here is meaningless, and other searchers all have different tradeoffs, so it seems unwise for the standard to specify one as the default choice.

§ 4.9. Future Extensions

§ 4.9.1. Parallel Overloads

After C++26 adopted [P3179R9], most of the algorithms in the `std::ranges` namespace gained a parallel overload that accepts an execution policy as the first argument. Since searchers are born to increase the performance of searching, it seems natural to also add parallel overload to them to further increase efficiency.

However, most existing specialized algorithms are designed for sequential usage. Paralleled versions of the Boyer-Moore algorithm are certainly [possible](#), but seem to require non-trivial handling of edge cases, such as when the pattern crosses the slicing boundaries. Given that C++17's parallel STL library deliberately did not add an execution policy overload to the searcher version of `std::search`, this proposal also does not add an execution policy overload to `std::ranges::search` or `std::ranges::contains_subrange`.

A future parallel extension probably requires some kind of coordination between the algorithm and the searcher, so both the constructor and `operator()` of the searcher probably should accept an execution policy argument (which also requires a new version of the concept `std::searchable`). However, such extensions are outside the scope of this proposal.

§ 4.9.2. Searcher Overloads In More Algorithms

Currently, searchers are only accepted and used in `std::search`. This proposal does a slight extension in this regard to make `std::ranges::contains_subrange` accept searchers too.

In the future, it can be envisioned that more algorithms can take advantage of searchers. For instance, `search_n` may also accept a searcher that is constructed from `count` and `value`, and `find` family may also benefit from more performant algorithms that search for single elements. However, such extensions are outside the scope of this proposal.

§ 4.9.3. More Searchers

Finally, there are various attempts to add more searchers to the standard library, providing users with searchers that satisfy more tradeoff decisions. Popular choices include the famous [Knuth-Morris-Pratt String-Searching Algorithm](#) and [Crochemore-Perrin Search Algorithm](#) ([P0638R0]). Similar to Range algorithms, future extensions to searchers probably should only add types to `std::ranges` (in particular `crochemore_perrin_searcher` from [P0638R0] can use `std::ranges::less` instead). Though the author will not oppose adding `std::` versions too.

§ 5. Implementation Experience

The author implemented this proposal in [beman.range_searcher](#) as part of the Beman Project. The implementation basically copies libc++'s searcher implementation and modify them to accommodate the new API. All modifications are straightforward, and no significant obstacles are observed.

§ 6. Questions To Resolve

§ 6.1. Questions for SG9

- Should the searchers be templated on range types or iterator/sentinel types?

§ 6.2. Questions for LEWG

- `std::search` does specify that the searchers don't need to satisfy *Cpp17CopyConstructible*; do we want to relax the `copyable` requirement in `searchable`?

- Should searchers' `operator()` be SFINAE-friendly (i.e., put the same value type requirements in the `requires` clause)?

§ 7. Wording

The wording below is based on [\[N5032\]](#).

§ 7.1. 17.3.2 Header `<version>` synopsis [version.syn]

In this clause's synopsis, make the following modification, substituting `20XXYYL` by the date of adoption. All inserted new macro definition should be in places that respects the current alphabetical order of the synopsis.

```
#define __cpp_lib_ranges_contains 20XXYYL202207L // freestanding, also in <algorithm>
// [...]
#define __cpp_lib_ranges_search 20XXYYL // freestanding, also in <algorithm>
#define __cpp_lib_ranges_searcher 20XXYYL // freestanding, also in <functional>
```

§ 7.2. 22.10.2 Header `<functional>` synopsis [functional.syn]

Modify the synopsis as follows:

```
// [...]
namespace std {
    // [...]
    // [func.search], searchers
    // [...]
    template<class RandomAccessIterator,
             class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
             class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher;

    namespace ranges {
        template <forward_range R,
                 copy_constructible Pred = equal_to,
                 indirectly_regular_unary_invocable<iterator t<R>> Proj = identity>
        class default_searcher; // freestanding
```

```

template <random_access_range R,
         copy_constructible Pred = equal_to,
         indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity,
         class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
         requires semiregular<range_value_t<R>>
class boyer_moore_searcher;

template <random_access_range R,
         copy_constructible Pred = equal_to,
         indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity,
         class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
         requires semiregular<range_value_t<R>>
class boyer_moore_horspool_searcher;
}
// [...]
}

```

Editor's Note: Insert the following subclause at the end of 22.10.18 Searchers [func.search]

§ 7.3. 22.10.18.5 Range-based searchers [func.range.search]

§ 7.3.1. 22.10.18.5.1 Class template `default_searcher` [func.range.search.default]

```

namespace std::ranges {
    template <forward_range R,
             copy_constructible Pred = equal_to,
             indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity,
             class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
    class default_searcher {
    public:
        constexpr default_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_last,
                                   Pred pred = {}, Proj proj = {});
        template<typename R2> requires same_as<remove_reference_t<R2>, R>
        constexpr explicit default_searcher(R2&& r);
        template<typename R2> requires same_as<remove_reference_t<R2>, R>
        constexpr default_searcher(R2&& r, Pred pred, Proj proj = {});

        template<forward_iterator I2, sentinel_for<I2> S2, class Proj2 = identity,
                copy_constructible Pred2 = equal_to,
                indirectly_regular_unary_invocable<iterator_t<I2>> Proj2 = identity>
        constexpr subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = {},
                                           Pred2 pred2 = {}, Proj2 proj2 = {});
        requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj2>
    };
}

```

```

template<forward_range R2, class Proj2 = identity>
constexpr borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = Proj2(),
requires indirectly_comparable<iterator_t<R2>, iterator_t<R>>, Pred = Pred())
{
private:
    [[no_unique_address]] std::ranges::iterator_t<R> pat_first_; // exposition
    [[no_unique_address]] std::ranges::sentinel_t<R> pat_last_; // exposition
    [[no_unique_address]] Pred pred_; // exposition
    [[no_unique_address]] Proj proj_; // exposition
}

template <forward_range R,
copy_constructible Pred = equal_to,
indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity>
default_searcher(R&&, Pred = {}, Proj = {})
-> default_searcher<remove_reference_t<R>, Pred, Proj>
}

```

```

constexpr default_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_last,
Pred pred = {}, Proj proj = {});

```

1. *Effects*: Constructs a `default_searcher` object, initializing `pat_first_` with `pat_first`, `pat_last_` with `pat_last`, `pred_` with `move(pred)`, and `proj_` with `move(proj)`.
2. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`.

```

template<typename R2> requires same_as<remove_reference_t<R2>, R>
constexpr explicit default_searcher(R2&& r);

```

3. *Effects*: Constructs a `default_searcher` object, initializing `pat_first_` with `ranges::begin(r)` and `pat_last_` with `ranges::end(r)`.
4. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`.

```

template<typename R2> requires same_as<remove_reference_t<R2>, R>
constexpr default_searcher(R2&& r, Pred pred, Proj proj = {});

```

5. *Effects*: Constructs a `default_searcher` object, initializing `pat_first_` with `ranges::begin(r)`, `pat_last_` with `ranges::end(r)`, `pred_` with `move(pred)`, and

`proj_` with `move(proj)`.

6. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`.

```
template<forward_iterator I2, sentinel_for<I2> S2, class Proj2 = identity>
constexpr subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = {}) const
requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj>;
```

7. *Effects*: Equivalent to: `return ranges::search(first, last, pat_first_, pat_last_, pred_, move(proj2), proj_);`

```
template<forward_range R2, class Proj2 = identity>
constexpr borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = {}) const
requires indirectly_comparable<iterator_t<R2>, iterator_t<R>, Pred, Proj>;
```

8. *Effects*: Equivalent to: `return ranges::search(ranges::begin(r2), ranges::end(r2), pat_first_, pat_last_, pred_, move(proj2), proj_);`

§ 7.3.2. 22.10.18.5.2 Class template `boyer_moore_searcher` [`func.range.search.bm`]

```
namespace std::ranges {
    template <random_access_range R,
              copy_constructible Pred = equal_to,
              indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity,
              class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
    requires semiregular<range_value_t<R>>
    class boyer_moore_searcher {
    public:
        boyer_moore_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_last,
                             Pred pred = {}, Proj proj = {}, Hash hf = {});
        template<typename R2> requires same_as<remove_reference_t<R2>, R>
        explicit boyer_moore_searcher(R2&& r);
        template<typename R2> requires same_as<remove_reference_t<R2>, R>
        boyer_moore_searcher(R2&& r, Pred pred, Proj proj = {}, Hash hf = {});

        template<random_access_iterator I2, sentinel_for<I2> S2, class Proj2 = identity>
        subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = {}) const
        requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj>;
        template<random_access_range R2, class Proj2 = identity>
```

```

        borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = {}) const
            requires indirectly_comparable<iterator_t<R2>, iterator_t<R>, Pred>

private:
    [[no_unique_address]] std::ranges::iterator_t<R> pat_first_; // exposi
    [[no_unique_address]] std::ranges::sentinel_t<R> pat_last_; // exposi
    [[no_unique_address]] Pred pred_; // exposi
    [[no_unique_address]] Proj proj_; // exposi
    [[no_unique_address]] Hash hash_; // exposi
}

template <random_access_range R,
          copy_constructible Pred = equal_to,
          indirectly_regular_unary_invocable<iterator_t<R>> Proj = ident:
          class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
    requires semiregular<range_value_t<R>>
    boyer_moore_searcher(R&&, Pred = {}, Proj = {}, Hash = {})
        -> boyer_moore_searcher<remove_reference_t<R>, Pred, Proj, Hash>;
}

```

```

boyer_moore_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_last,
                    Pred pred = {}, Proj proj = {}, Hash hf = {});

```

1. *Preconditions:* Let `V` be `range_value_t<R>`. For any two values `A` and `B` of type `V`, if `pred(invoker(proj, A), invoker(proj, B)) == true`, then `hf(invoker(proj, A)) == hf(invoker(proj, B))` is true.
2. *Effects:* Constructs a `boyer_moore_searcher` object, initializing `pat_first_` with `pat_first`, `pat_last_` with `pat_last`, `pred_` with `move(pred)`, `proj_` with `move(proj)`, and `hash_` with `move(hf)`.
3. *Throws:* Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `R`, or `operator()` of `Pred` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```

template<typename R2> requires same_as<remove_reference_t<R2>, R>
    explicit boyer_moore_searcher(R2&& r);

```

4. *Preconditions:* Let `V` be `range_value_t<R>`. For any two values `A` and `B` of type `V`, if `Pred{}(invoker(proj, A), invoker(proj, B)) == true`, then `Hash{}(invoker(proj,`

A)) == Hash{}(invoke(proj, B)) is true.

5. *Effects*: Constructs a `boyer_moore_searcher` object, initializing `pat_first_` with `ranges::begin(r)` and `pat_last_` with `ranges::end(r)`.

6. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `R`, or `operator()` of `Pred` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```
template<typename R2> requires same_as<remove_reference_t<R2>, R>
boyer_moore_searcher(R2&& r, Pred pred, Proj proj = {}, Hash hf = {});
```

7. *Preconditions*: Let `V` be `range_value_t<R>`. For any two values `A` and `B` of type `V`, if `pred(invoke(proj, A), invoke(proj, B)) == true`, then `hf(invoke(proj, A)) == hf(invoke(proj, B))` is true.

8. *Effects*: Constructs a `boyer_moore_searcher` object, initializing `pat_first_` with `ranges::begin(r)`, `pat_last_` with `ranges::end(r)`, `pred_` with `move(pred)`, `proj_` with `move(proj)`, and `hash_` with `move(hf)`.

9. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `R`, or `operator()` of `Pred` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```
template<random_access_iterator I2, sentinel_for<I2> S2, class Proj2 = identity,
subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = {}) const
requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj>;
```

10. *Mandates*: `is_same_v<remove_cvref_t<projected_value_t<iterator_t<R>, Proj>>, remove_cvref_t<projected_value_t<I2, Proj2>>>` is true.

11. *Effects*: Finds a subsequence of equal values in a sequence.

12. *Returns*: `subrange(i, j)`, where `i` and `j` is a pair of iterators such that

- `i` is the first iterator in the range `[first, last - (pat_last_ - pat_first_))` such that for every non-negative integer `n` less than `pat_last_ - pat_first_` the following condition holds: `pred_(invoke(proj2, *(i + n)), invoke(proj_, *(pat_first_ + n))) != false`, and
- `j == ranges::next(i, ranges::distance(pat_first_, pat_last_))`.

Returns `subrange(first, first)` if `[pat_first_, pat_last_)` is empty, otherwise returns `subrange(last, last)` if no such iterator is found.

13. *Complexity*: At most $(last - first) * (pat_last_ - pat_first_)$ applications of the predicate, and at most $(last - first) * (pat_last_ - pat_first_ + 1)$ applications of either projection.

```
template<random_access_range R2, class Proj2 = identity>
    borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = {}) const
        requires indirectly_comparable<iterator_t<R2>, iterator_t<R>, Pred, Proj2>
```

14. *Effects*: Equivalent to: `return operator()(ranges::begin(r2), ranges::end(r2), move(proj2));`

§ 7.3.3. 22.10.18.5.3 Class template `boyer_moore_horspool_searcher` [`func.range.search.bmh`]

```
namespace std::ranges {
    template <random_access_range R,
              copy_constructible Pred = equal_to,
              indirectly_regular_unary_invocable<iterator_t<R>> Proj = identity,
              class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
        requires semiregular<range_value_t<R>>
        class boyer_moore_horspool_searcher {
        public:
            boyer_moore_horspool_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_last,
                                           Pred pred = {}, Proj proj = {}, Hash hf = {})
            template<typename R2> requires same_as<remove_reference_t<R2>, R>
            explicit boyer_moore_horspool_searcher(R2&& r);
            template<typename R2> requires same_as<remove_reference_t<R2>, R>
            boyer_moore_horspool_searcher(R2&& r, Pred pred, Proj proj = {}, Hash hf = {});

            template<random_access_iterator I2, sentinel_for<I2> S2, class Proj2 = identity>
                subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = {}) const
                    requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj>
            template<random_access_range R2, class Proj2 = identity>
                borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = {}) const
                    requires indirectly_comparable<iterator_t<R2>, iterator_t<R>, Pred, Proj2>

        private:
            [[no_unique_address]] std::ranges::iterator_t<R> pat_first_; // exposition only
        };
    };
}
```

```

[[no_unique_address]] std::ranges::sentinel_t<R> pat_last_; // exposi
[[no_unique_address]] Pred pred_; // exposi
[[no_unique_address]] Proj proj_; // exposi
[[no_unique_address]] Hash hash_; // exposi
}

template <random_access_range R,
         copy_constructible Pred = equal_to,
         indirectly_regular_unary_invocable<iterator_t<R>> Proj = identit
         class Hash = hash<projected_value_t<iterator_t<R>, Proj>>>
requires semiregular<range_value_t<R>>
boyer_moore_horspool_searcher(R&&, Pred = {}, Proj = {}, Hash = {})
    -> boyer_moore_horspool_searcher<remove_reference_t<R>, Pred, Proj, Hash>
}

```

```

boyer_moore_horspool_searcher(iterator_t<R> pat_first, sentinel_t<R> pat_la
                             Pred pred = {}, Proj proj = {}, Hash hf = {});

```

1. *Preconditions:* Let V be `range_value_t<R>`. For any two values A and B of type V , if `pred(invoker(proj, A), invoker(proj, B)) == true`, then `hf(invoker(proj, A)) == hf(invoker(proj, B))` is true.
2. *Effects:* Constructs a `boyer_moore_horspool_searcher` object, initializing `pat_first_` with `pat_first`, `pat_last_` with `pat_last`, `pred_` with `move(pred)`, `proj_` with `move(proj)`, and `hash_` with `move(hf)`.
3. *Throws:* Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of R , or `operator()` of `Pred` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```

template<typename R2> requires same_as<remove_reference_t<R2>, R>
explicit boyer_moore_horspool_searcher(R2&& r);

```

4. *Preconditions:* Let V be `range_value_t<R>`. For any two values A and B of type V , if `Pred{}(invoker(proj, A), invoker(proj, B)) == true`, then `Hash{}(invoker(proj, A)) == Hash{}(invoker(proj, B))` is true.
5. *Effects:* Constructs a `boyer_moore_horspool_searcher` object, initializing `pat_first_` with `ranges::begin(r)` and `pat_last_` with `ranges::end(r)`.

6. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `R`, or `operator()` of `Pred` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```
template<typename R2> requires same_as<remove_reference_t<R2>, R>
boyer_moore_horspool_searcher(R2&& r, Pred pred, Proj proj = {}, Hash hf
```

7. *Preconditions*: Let `V` be `range_value_t<R>`. For any two values `A` and `B` of type `V`, if `pred(invoker(proj, A), invoker(proj, B)) == true`, then `hf(invoker(proj, A)) == hf(invoker(proj, B))` is true.

8. *Effects*: Constructs a `boyer_moore_horspool_searcher` object, initializing `pat_first_` with `ranges::begin(r)`, `pat_last_` with `ranges::end(r)`, `pred_` with `move(pred)`, `proj_` with `move(proj)`, and `hash_` with `move(hf)`.

9. *Throws*: Any exception thrown by the copy constructor of `iterator_t<R>` or `sentinel_t<R>`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `R`, or `operator()` of `Pred` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```
template<random_access_iterator I2, sentinel_for<I2> S2, class Proj2 = identity,
subrange<I2> operator()(I2 first, S2 last, Proj2 proj2 = {}) const
requires indirectly_comparable<I2, iterator_t<R>, Pred, Proj2, Proj>;
```

10. *Mandates*: `is_same_v<remove_cvref_t<projected_value_t<iterator_t<R>, Proj>>, remove_cvref_t<projected_value_t<I2, Proj2>>>` is true.

11. *Effects*: Finds a subsequence of equal values in a sequence.

12. *Returns*: `subrange(i, j)`, where `i` and `j` is a pair of iterators such that

- `i` is the first iterator in the range `[first, last - (pat_last_ - pat_first_))` such that for every non-negative integer `n` less than `pat_last_ - pat_first_` the following condition holds: `pred(invoker(proj2, *(i + n)), invoker(proj_, *(pat_first_ + n))) != false`, and
- `j == ranges::next(i, ranges::distance(pat_first_, pat_last_))`.

Returns `subrange(first, first)` if `[pat_first_, pat_last_)` is empty, otherwise returns `subrange(last, last)` if no such iterator is found.

13. *Complexity*: At most $(last - first) * (pat_last_ - pat_first_)$ applications of the predicate, and at most $(last - first) * (pat_last_ - pat_first_ + 1)$ applications of either projection.

```
template<random_access_range R2, class Proj2 = identity>
    borrowed_subrange_t<R2> operator()(R2&& r2, Proj2 proj2 = {}) const
        requires indirectly_comparable<iterator_t<R2>, iterator_t<R>, Pred, Proj2>
```

14. *Effects*: Equivalent to: `return operator()(ranges::begin(r2), ranges::end(r2), move(proj2));`

§ 7.4.24.2 Header `<iterator>` synopsis [iterator.synopsis]

Modify the synopsis as follows:

```
// [...]
namespace std {
    // [...]
    // [alg.req.ind.cmp], concept indirectly_comparable
    template<class I1, class I2, class R, class P1 = identity, class P2 = identity>
    concept indirectly_comparable = see below; // freestanding

    // [alg.req.searchable], concept searchable
    template<class Searcher, class I, class S, class P = identity>
    concept searchable = see below; // freestanding

    // [alg.req.permutable], concept permutable
    template<class I>
    concept permutable = see below; // freestanding
    // [...]
}
```

Editor's Note: Insert the following subclause between 24.3.7.5 Concept `indirectly_comparable` [alg.req.ind.cmp] and 24.3.7.6 Concept `permutable` [alg.req.permutable]

§ 7.5.24.3.7. Concept `searchable` [alg.req.searchable]

1. The `searchable` concept specifies the requirements of algorithms that interact with searchers (22.10.18 [func.search]) to search for a sequence in another sequence.

```
template<class Searcher, class I, class S, class P = identity>
concept searchable =
    copyable<Searcher> &&
    invocable<Searcher, I, S, P> &&
    ranges::forward_range<invoke_result_t<Searcher, I, S, P>>
```

§ 7.6.26.4 Header `<algorithm>` synopsis [algorithm.syn]

Modify the synopsis as follows:

```
// [...]
namespace std {
    // [...]
    // [alg.contains], contains
    namespace ranges {
        // [...]
        template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
                class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred>,
        bool contains_subrange(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {}, Proj1 proj1 = {},
                               Proj2 proj2 = {}); // freestanding-deleted

        template<forward_iterator I, sentinel_for<I> S,
                class Searcher, class Proj = identity>
        requires searchable<Searcher, I, S, Proj>
        constexpr bool contains_subrange(I first, S last,
                                         const Searcher& searcher, Proj proj);

        template<forward_range R, class Searcher, class Proj = identity>
        requires searchable<Searcher, iterator_t<R>, sentinel_t<R>, Proj>
        constexpr bool contains_subrange(R&& r, const Searcher& searcher, Proj proj);
    }
    // [...]
    // [alg.search], search
    // [...]
    template<class ForwardIterator, class Searcher>
```

```
constexpr ForwardIterator
search(ForwardIterator first, ForwardIterator last, const Searcher& s) {
    namespace ranges {
        template<forward_iterator I, sentinel_for<I> S,
                class Searcher, class Proj = identity>
        requires searchable<Searcher, I, S, Proj>
        constexpr auto search(I first, S last,
                               const Searcher& searcher, Proj proj = {});
        template<forward_range R, class Searcher, class Proj = identity>
        requires searchable<Searcher, iterator_t<R>, sentinel_t<R>, Proj>
        constexpr auto search(R&& r, const Searcher& searcher, Proj proj = {});
    }
    // [...]
}
```

§ 7.7.26.6.4 Contains [alg.contains]

Add to the end of this clause:

```
template<forward_iterator I, sentinel_for<I> S,
        class Searcher, class Proj = identity>
requires searchable<Searcher, I, S, Proj>
constexpr bool ranges::contains_subrange(I first, S last,
                                          const Searcher& searcher, Proj proj)
```

5. *Returns:*

```
!ranges::empty(ranges::search(first, last, searcher, move(proj)))
```

```
template<forward_range R, class Searcher, class Proj = identity>
requires searchable<Searcher, iterator_t<R>, sentinel_t<R>, Proj>
constexpr bool ranges::contains_subrange(R&& r, const Searcher& searcher,
```

6. *Returns:*

```
!ranges::empty(ranges::search(forward<R>(r), searcher, move(proj)))
```

§ 7.8.26.6.15 Search [alg.search]

Add to the end of this clause:

```
template<forward_iterator I, sentinel_for<I> S,  
        class Searcher, class Proj = identity>  
    requires searchable<Searcher, I, S, Proj>  
    constexpr auto ranges::search(I first, S last,  
                                  const Searcher& searcher, Proj proj = {});
```

13. *Effects*: Equivalent to: `return searcher(first, last, move(proj));`

```
template<forward_range R, class Searcher, class Proj = identity>  
    requires searchable<Searcher, iterator_t<R>, sentinel_t<R>, Proj>  
    constexpr auto ranges::search(R&& r, const Searcher& searcher, Proj proj
```

14. *Effects*: Equivalent to:

- `return searcher(forward<R>(r), move(proj))` if that expression is well-formed.
- Otherwise, `return searcher(ranges::begin(r), ranges::end(r), move(proj));`.

§ References

§ Normative References

[N3905]

Marshall Clow. *Extending std::search to use Additional Searching Algorithms (Version 4)*. 14 February 2014. URL: <https://wg21.link/n3905>

[N5032]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025. URL: <https://wg21.link/n5032>

§ Informative References

[LWG4286]

Yihe Li. *Some more feature-test macros for fully freestanding features are not marked freestanding*. WP. URL: <https://wg21.link/lwg4286>

[P0202R3]

Antony Polukhin. *Add Constexpr Modifiers to Functions in <algorithm> and <utility> Headers*. 9 November 2017. URL: <https://wg21.link/p0202r3>

[P0253R1]

Marshall Clow. *Fixing a design mistake in the searchers interface in Library Fundamentals*. 1 March 2016. URL: <https://wg21.link/p0253r1>

[P0638R0]

Ed Schouten. *Crochemore-Perrin search algorithm for std::search*. 3 May 2017. URL: <https://wg21.link/p0638r0>

[P0896R4]

Eric Niebler, Casey Carter, Christopher Di Bella. *The One Ranges Proposal*. 9 November 2018. URL: <https://wg21.link/p0896r4>

[P1642R11]

Ben Craig. *Freestanding Library: Easy [utilities], [ranges], and [iterators]*. 1 July 2022. URL: <https://wg21.link/p1642r11>

[P1813R0]

Christopher Di Bella. *A Concept Design for the Numeric Algorithms*. 2 August 2019. URL: <https://wg21.link/p1813r0>

[P2198R7]

Ben Craig. *Freestanding Feature-Test Macros and Implementation-Defined Extensions*. 14 December 2022. URL: <https://wg21.link/p2198r7>

[P2302R4]

Christopher Di Bella. *std::ranges::contains*. 17 April 2022. URL: <https://wg21.link/p2302r4>

[P2976R1]

Ben Craig. *Freestanding Library: algorithm, numeric, and random*. 5 May 2024. URL: <https://wg21.link/p2976r1>

[P3179R9]

Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. *C++ parallel range algorithms*. 29 May 2025. URL: <https://wg21.link/p3179r9>

[P3732R2]

Ruslan Arutyunyan, Mark Hoemmen, Alexey Kukanov, Bryce Adelstein Lelbach, Abhilash Majumder. *Numeric Range Algorithms*. 26 March 2026. URL: <https://wg21.link/p3732r2>